

UNIVERSIDAD POLITÉCNICA SALESIANA

Ingeniería en Ciencias de la Computación

INFORME

Ping Pong en Red

Asignatura:	Sistemas Distribuidos
Docente:	Julio Proaño
Nivel / paralelo:	6to Semestre / Grupo 1
Integrantes:	Ian Mena Juan Núñez Francisco Urgiles
Fecha de entrega:	23/10/26
Período académico:	69
Sede:	Quito

1. Introducción

El presente proyecto consiste en una versión distribuida del clásico juego Pong, desarrollada en Python con la biblioteca pygame. El sistema sigue una arquitectura cliente-servidor con servidor autoritativo: el servidor calcula toda la física del juego (movimiento de la pelota, rebotes, cambios de color y marcador), mientras que cada cliente se limita a enviar las teclas del jugador y a dibujar el estado que recibe. La comunicación se realiza mediante sockets TCP, con un protocolo propio basado en mensajes JSON definido en el módulo protocolo.py. El código se organiza en tres archivos con responsabilidades separadas: protocolo.py (el contrato compartido), servidor.py (la lógica del juego y la gestión de conexiones) y cliente.py (la entrada del jugador y la visualización).

2. Repositorio

Github: https://github.com/iansmena/SD_PingPongLan

3. Conclusiones

- Un servidor autoritativo garantiza la consistencia del estado. Como la física se calcula en un solo proceso, ambos jugadores ven siempre la misma posición de la pelota y el mismo marcador. Si cada cliente simulara el juego por su cuenta, pequeñas diferencias en el tiempo de cada frame habrían provocado desincronización. Además, un cliente no puede alterar la pelota ni el puntaje, porque solo puede enviar las teclas “arriba” y “abajo”.
- TCP transmite bytes, no mensajes, por lo que fue necesario diseñar un protocolo explícito. Un mismo recv() puede entregar varios mensajes juntos o un mensaje incompleto. Por eso cada mensaje termina en un salto de línea y se acumula en un buffer hasta estar completo. Definir los mensajes como clases y añadir un número de versión permitió modificar el juego desde un único archivo y detectar clientes desactualizados. Cada mensaje de estado ocupa unos 130 bytes, lo que equivale a unos 7,8 KB/s por cliente a 60 actualizaciones por segundo, un consumo bajo para una red local.
- La concurrencia y la tolerancia a fallos son tan importantes como la lógica del juego. El servidor atiende a varios clientes al mismo tiempo mediante hilos, y protege el estado compartido con un lock para evitar condiciones de carrera. Cuando un jugador se desconecta, el servidor no se detiene: pausa la partida y permite que otro cliente ocupe su lugar. Como limitación, el servidor sigue siendo un punto único de falla, y se identifican como mejoras futuras la autenticación de jugadores y la predicción del lado del cliente para reducir el efecto de la latencia.

Anexo A. Evidencias de ejecución

```
(.venv) PS C:\Users\iansm\Documents\Proyectos\SD_PingPongLan\codigo_base> py .\servidor.py
Servidor de Pong v1 en 192.168.1.4:5555  (Ctrl+C para detener)
[+] ('192.168.1.4', 60799) conectado como 'izq'
[-] Jugador 'izq' desconectado. Esperando reconexión...
[+] ('192.168.1.4', 60665) conectado como 'izq'
[+] ('192.168.1.4', 60556) conectado como 'der'
```

```
 (.venv) PS C:\Users\iansm\Documents\Proyectos\SD_PingPongLan\codigo_base> py .\cliente.py
 pygame-ce 2.5.8 (SDL 2.32.10, Python 3.15.0)
```

